

Terraform-01 Task Documentation

Objective

The objective of this task is to install Terraform, understand core Terraform commands, execute sample Terraform templates shown in the training video, and integrate a sample Terraform project with Jenkins.

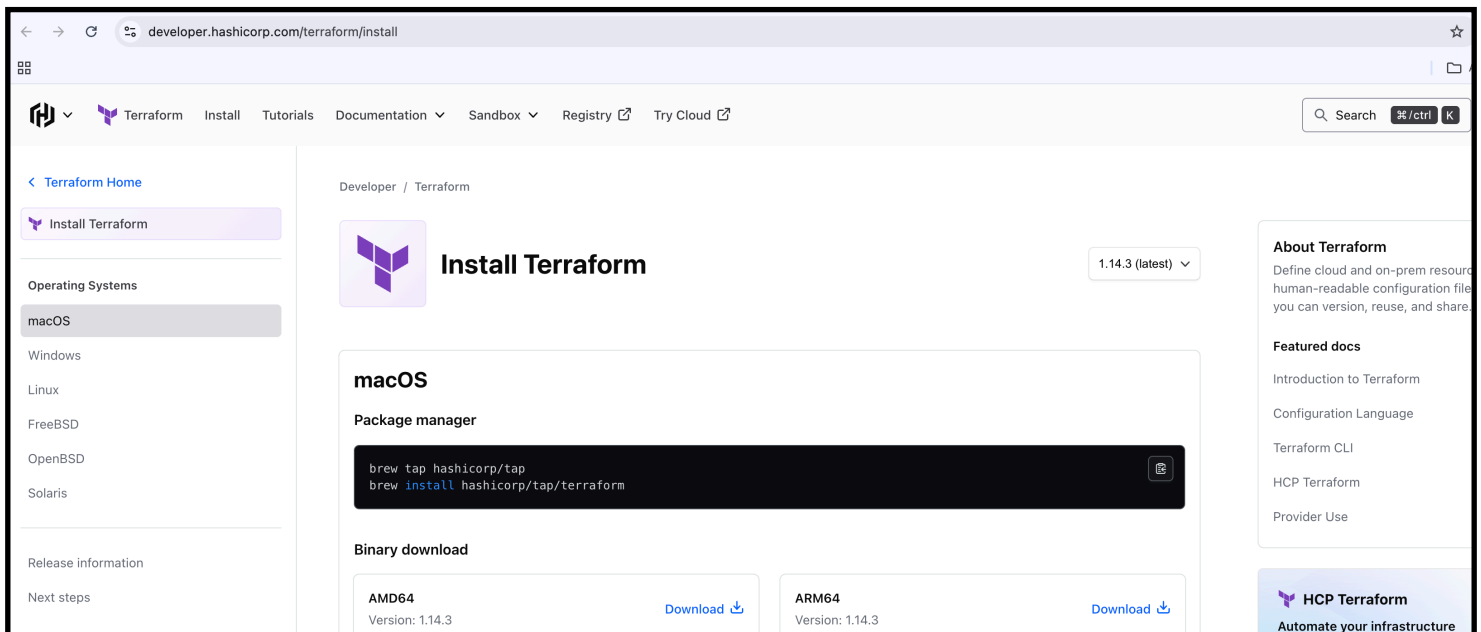
Task :1

1) Install Terraform on your PC

Step 1: Download

<https://developer.hashicorp.com/terraform/downloads>

Choose **macOS**



The screenshot shows the Terraform installation page for macOS. The page is titled "Install Terraform" and features a sidebar with navigation links. The main content area displays the "macOS" section, which includes a "Package manager" section with a code block for installing Terraform via Homebrew, and a "Binary download" section with links to download the AMD64 and ARM64 binaries. The right sidebar contains "About Terraform" information and "Featured docs" links.

developer.hashicorp.com/terraform/install

Terraform Home

Install Terraform

Operating Systems

macOS

Windows

Linux

FreeBSD

OpenBSD

Solaris

Release information

Next steps

Developer / Terraform

Install Terraform

1.14.3 (latest)

macOS

Package manager

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

Binary download

AMD64
Version: 1.14.3
Download

ARM64
Version: 1.14.3
Download

About Terraform

Define cloud and on-prem resources in a single, human-readable configuration file that you can version, reuse, and share.

Featured docs

Introduction to Terraform

Configuration Language

Terraform CLI

HCP Terraform

Provider Use

HCP Terraform

Automate your infrastructure

Now go to your local terminal add below commands

Step 2: Tap HashiCorp again (fresh metadata)

```
brew tap hashicorp/tap
```

◆ Step 3: Install latest Terraform

```
brew install hashicorp/tap/terraform
```

◆ Step 4: Final Verification

```
terraform -version
```

```
waseemakram@waseem desktop % brew tap hashicorp/tap

==> Tapping hashicorp/tap
Cloning into '/opt/homebrew/Library/Taps/hashicorp/homebrew-tap'...
remote: Enumerating objects: 6221, done.
remote: Counting objects: 100% (966/966), done.
remote: Compressing objects: 100% (241/241), done.
remote: Total 6221 (delta 849), reused 726 (delta 725), pack-reused 5255 (from 2)
Receiving objects: 100% (6221/6221), 1.08 MiB | 4.55 MiB/s, done.
Resolving deltas: 100% (4464/4464), done.
Tapped 2 casks and 32 formulae (99 files, 1.5MB).
waseemakram@waseem desktop % brew install hashicorp/tap/terraform

==> Fetching downloads for: terraform
✓ Formula terraform (1.14.3)
==> Installing terraform from hashicorp/tap
📦 /opt/homebrew/Cellar/terraform/1.14.3: 5 files, 95.6MB, built in 2 seconds
==> Running `brew cleanup terraform`...
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
Removing: /Users/waseemakram/Library/Caches/Homebrew/terraform--1.13.4.zip... (28.9MB)
waseemakram@waseem desktop % terraform -version

Terraform v1.14.3
on darwin_arm64
waseemakram@waseem desktop %
```

```
waseemakram@waseem desktop %
on darwin_arm64
Terraform v1.14.3

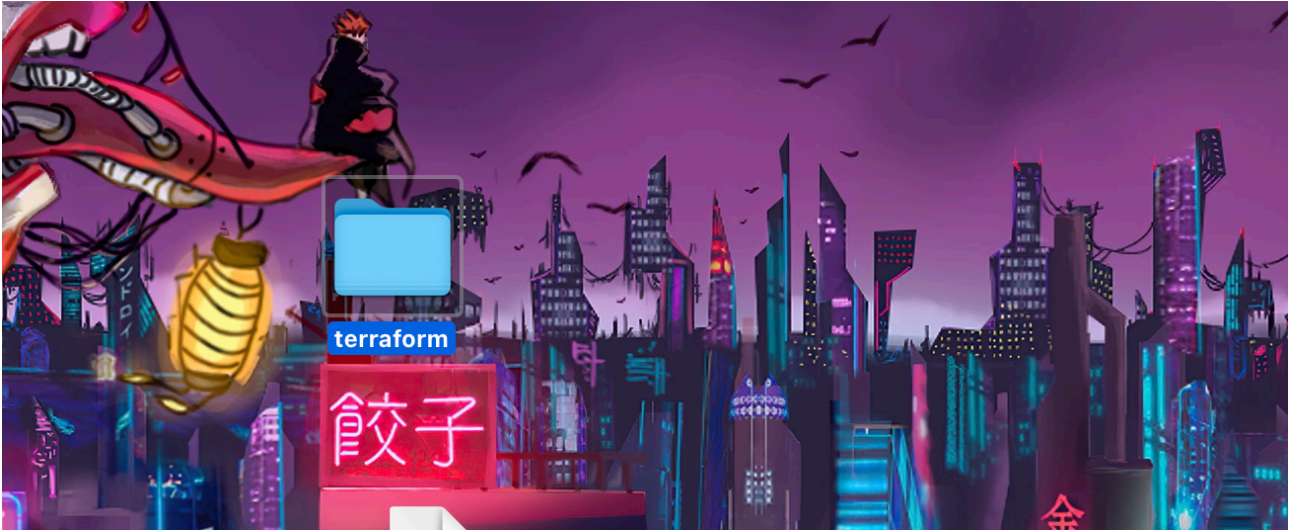
waseemakram@waseem desktop % terraform -version
Resolving: /opt/homebrew/Cellar/terraform/1.14.3: 5 files, 95.6MB, built in 2 seconds
waseemakram@waseem desktop %
```

Task 2:

2). Execute all the templates shown in video.

Step 1: Create Your First Terraform Project

—>Make a new folder for your project (example: terraform).

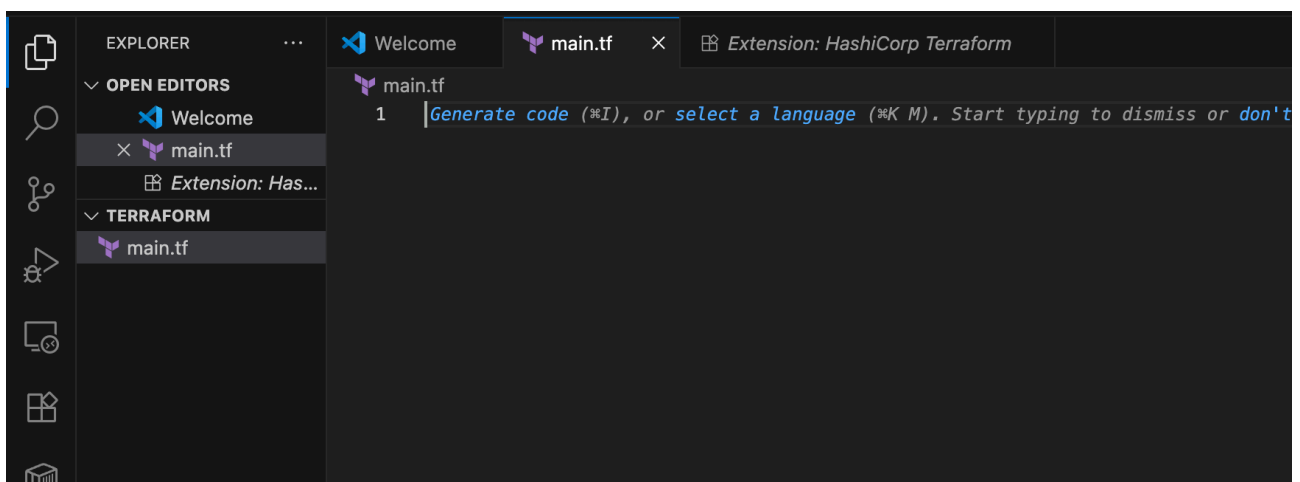


—>And open the folder in the vs code

—>Inside the folder , create your first Terraform file:

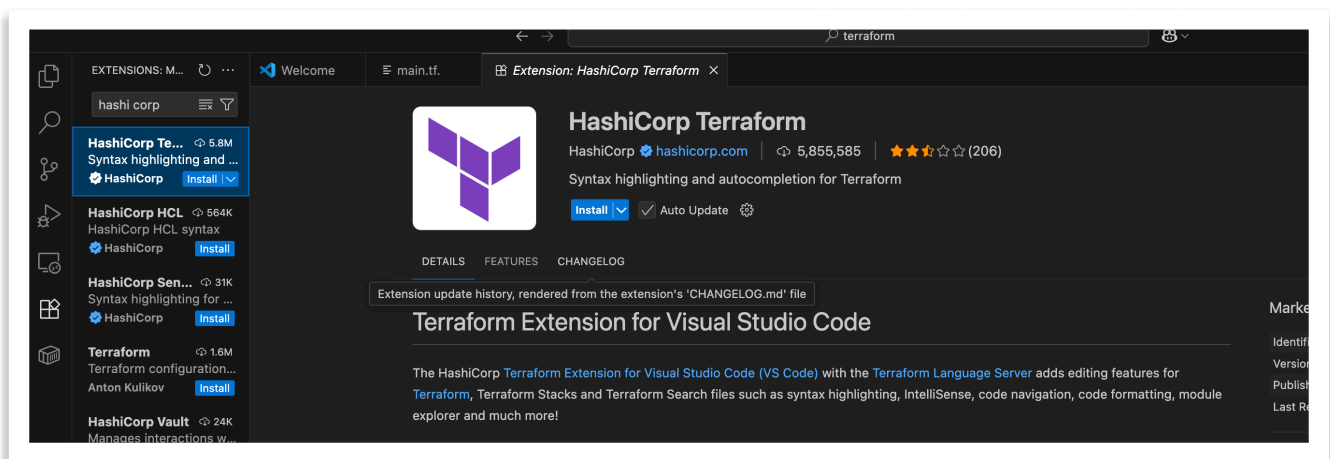
In VS Code → **File** → **New File** → **save as main.tf**.

Write a sample content in it



—> .tf —>is the extension for the terraform resource files

—> Make sure you have hashi corp tool installed

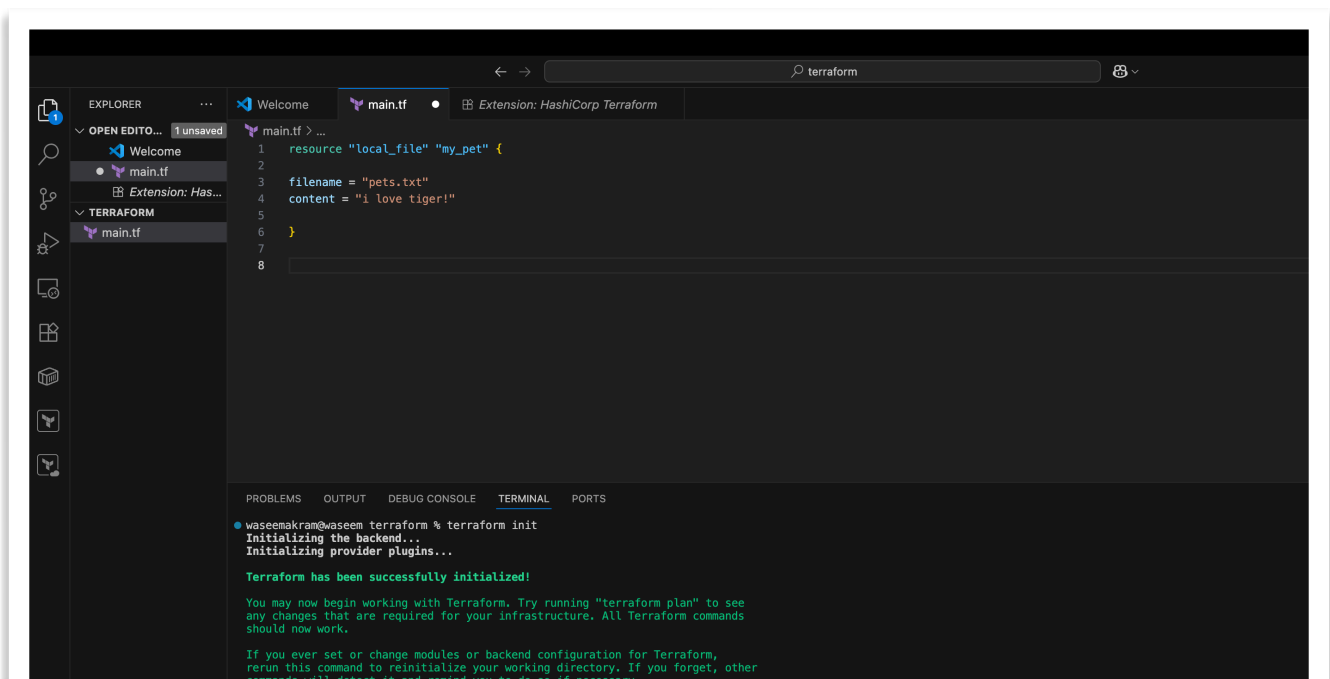


Install ✓

—> Write a sample content in it

```
resource "local_file" "my_pet" {  
  
  filename = "pets.txt"  
  content = "i love tiger!"  
}
```

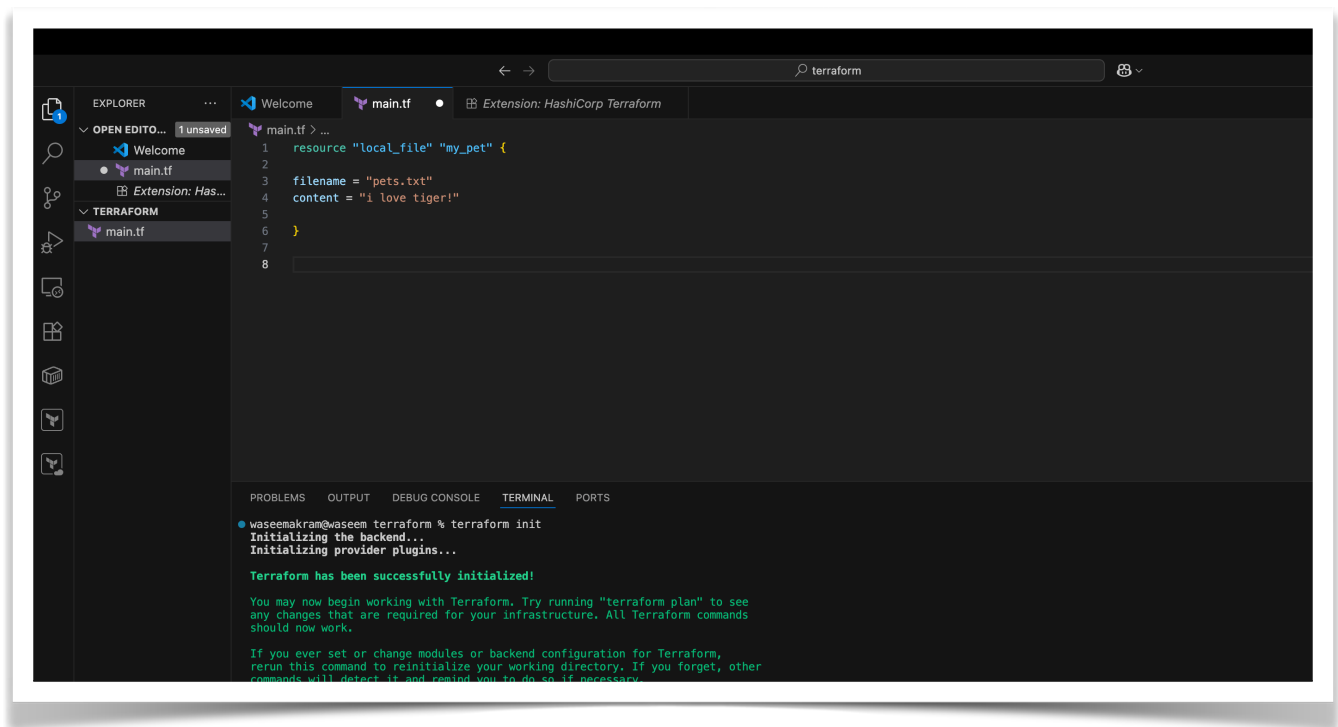
—>save



Step 2: Run Terraform Commands

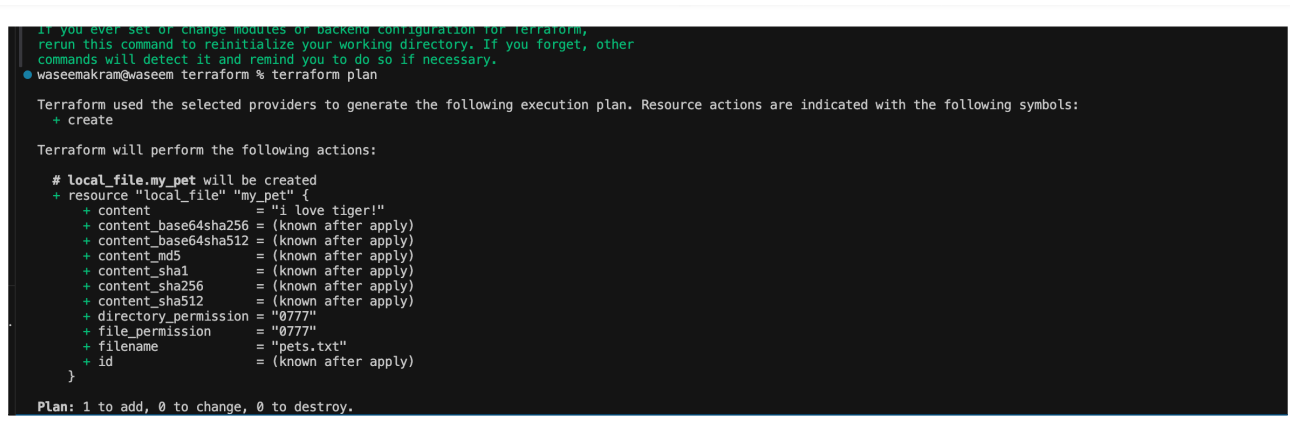
Initialize Terraform

terraform init



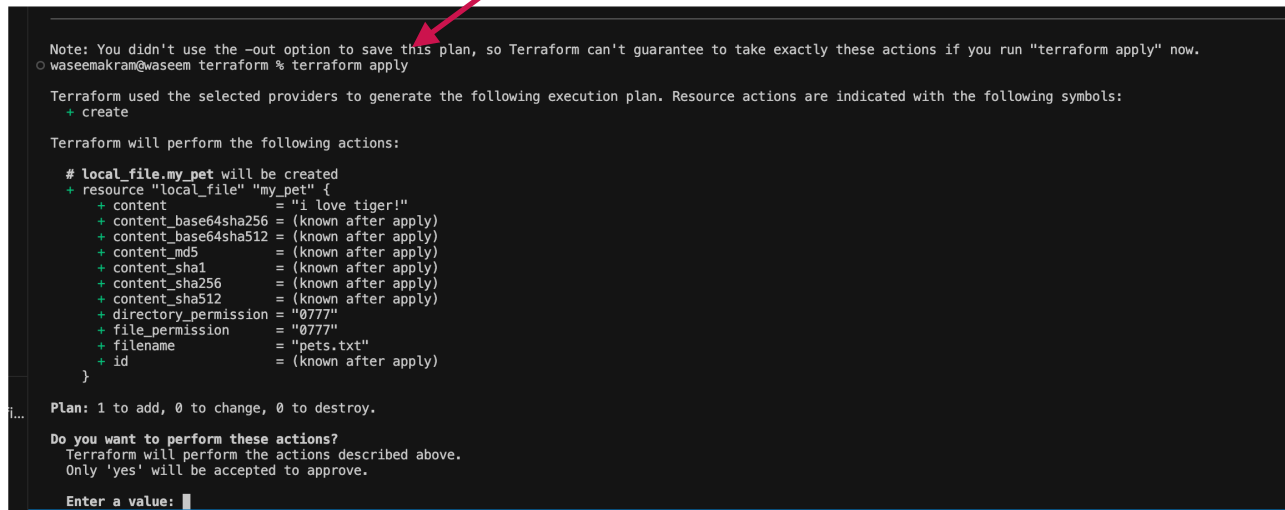
Now See what will be created:

terraform plan :



Apply changes (actually create resources):

terraform apply



```
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
waseemakram@waseem terraform % terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

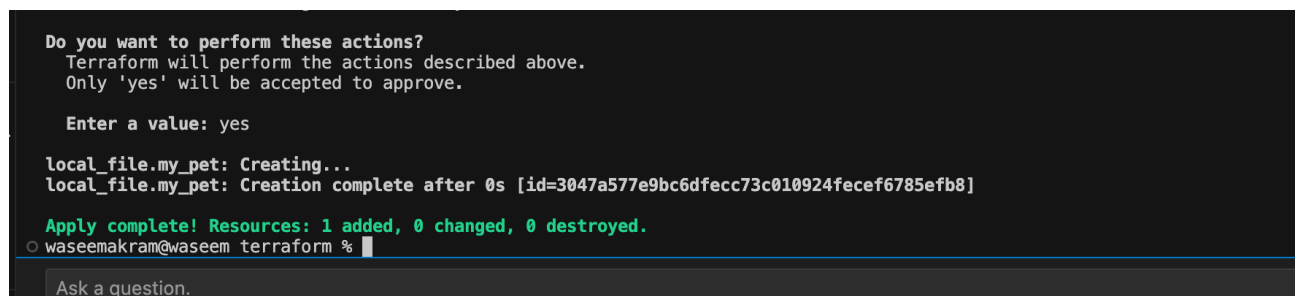
# local_file.my_pet will be created
+ resource "local_file" "my_pet" {
+   content          = "i love tiger!"
+   content_base64sha256 = (known after apply)
+   content_base64sha512 = (known after apply)
+   content_md5       = (known after apply)
+   content_sha1       = (known after apply)
+   content_sha256     = (known after apply)
+   content_sha512     = (known after apply)
+   directory_permission = "0777"
+   file_permission    = "0777"
+   filename          = "pets.txt"
+   id                 = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: 
```

Type yes when asked.

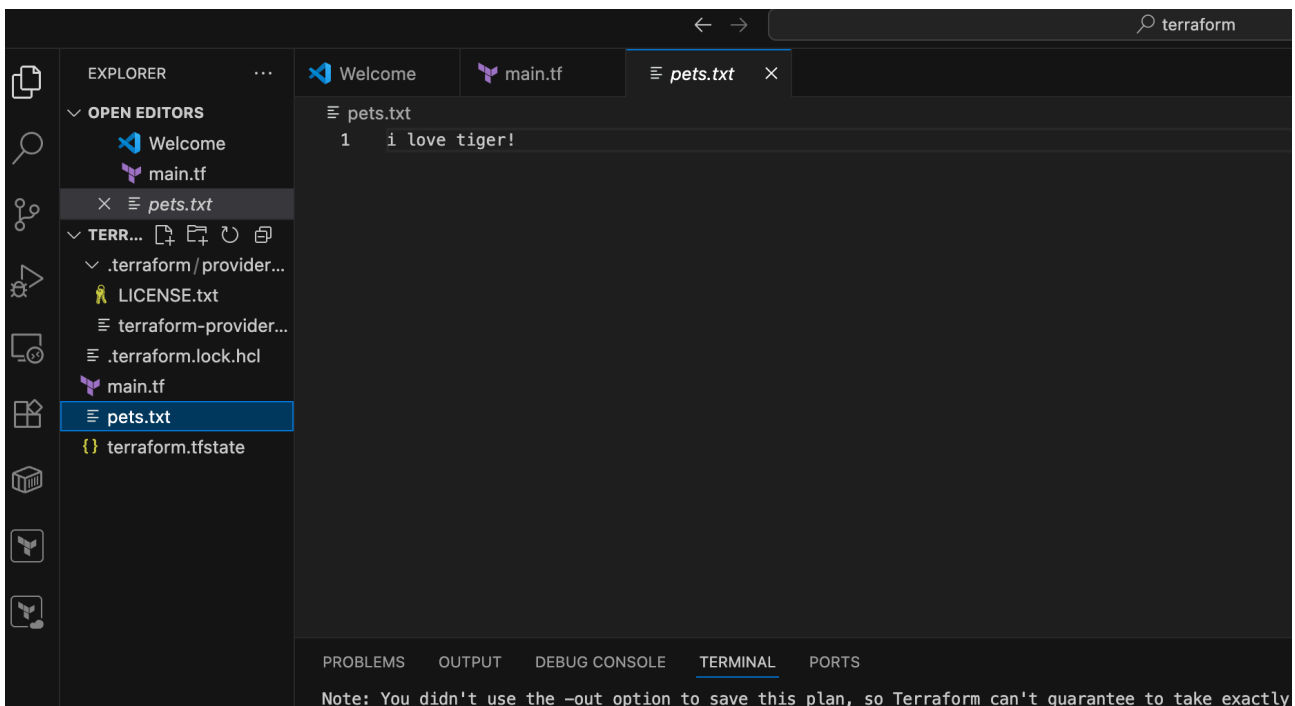


```
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=3047a577e9bc6dfec73c010924fecef6785efb8]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
waseemakram@waseem terraform % 
```



Destroy resources when finished:

terraform destroy

```
local_file.my_pet: creation complete after 0s [id=3047a577e9bc6dfec73c010924fecef6785efb8]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
waseemakram@waseem terraform % terraform destroy
local_file.my_pet: Refreshing state... [id=3047a577e9bc6dfec73c010924fecef6785efb8]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
- destroy

Terraform will perform the following actions:

# local_file.my_pet will be destroyed
- resource "local_file" "my_pet" {
  - content      = "i love tiger!" -> null
  - content_base64sha256 = "64NMjKd9RhYYRtpEW80nR0iV8XmTn/EQxjCsV2iQ8Rw==" -> null
  - content_base64sha512 = "04GiSpYFTyjzhvLQtI80IDD92fJqNyRlNAHxZGbk17j0049r+a0eEeUn3mgwkU08foQ6vyTMgXdajcJZ0YUWnw==" -> null
  - content_md5      = "1be4fcfb5b338961ca604c149be3dc22" -> null
  - content_sha1     = "3047a577e9bc6dfec73c010924fecef6785efb8" -> null
  - content_sha256   = "eb834c8ca77d46161846da445bcd27474895f179939ff110c630ac576890f11c" -> null
  - content_sha512   = "d381a24a96054f28f386f2d0b48f0e2030fdd9f26a3724653401f16466e48bb8ce3b8f6bf9a39e11e527de6830914f347e843abf24cc81775a8" -> null
  - directory_permission = "0777" -> null
  - file_permission     = "0777" -> null
  - filename            = "pets.txt" -> null
  - id                  = "3047a577e9bc6dfec73c010924fecef6785efb8" -> null
}

Plan: 0 to add, 0 to change, 1 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

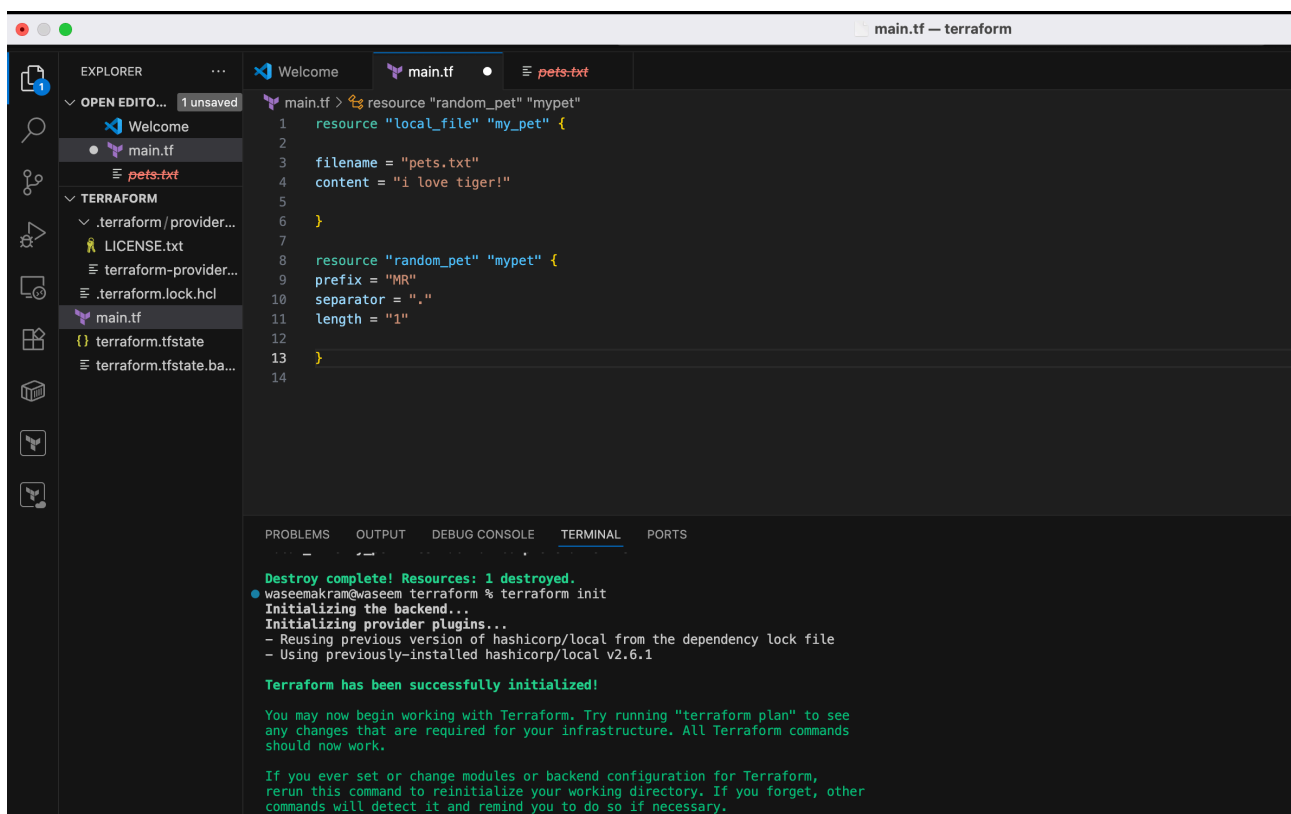
Enter a value: 
```

Now

→ created one more resource

Terraform init

apply



```
resource "local_file" "my_pet" {
```

```
  filename = "pets.txt"
  content  = "i love tiger!"
```

```
}
```

```
resource "random_pet" "mypet" {
```

```
  prefix      = "MR"
  separator   = "."
  length      = "1"
```

```
}
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
waseemakram@waseem terraform % terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# local_file.my_pet will be created
+ resource "local_file" "my_pet" {
  + content          = "i love tiger!"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_sha1      = (known after apply)
  + content_sha256     = (known after apply)
  + content_sha512     = (known after apply)
  + directory_permission = "0777"
  + file_permission    = "0777"
  + filename           = "pets.txt"
  + id                 = (known after apply)
}

# random_pet.mypet will be created
+ resource "random_pet" "mypet" {
  + id          = (known after apply)
  + length      = 1
  + prefix      = "MR"
  + separator   = "."
}

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.
```

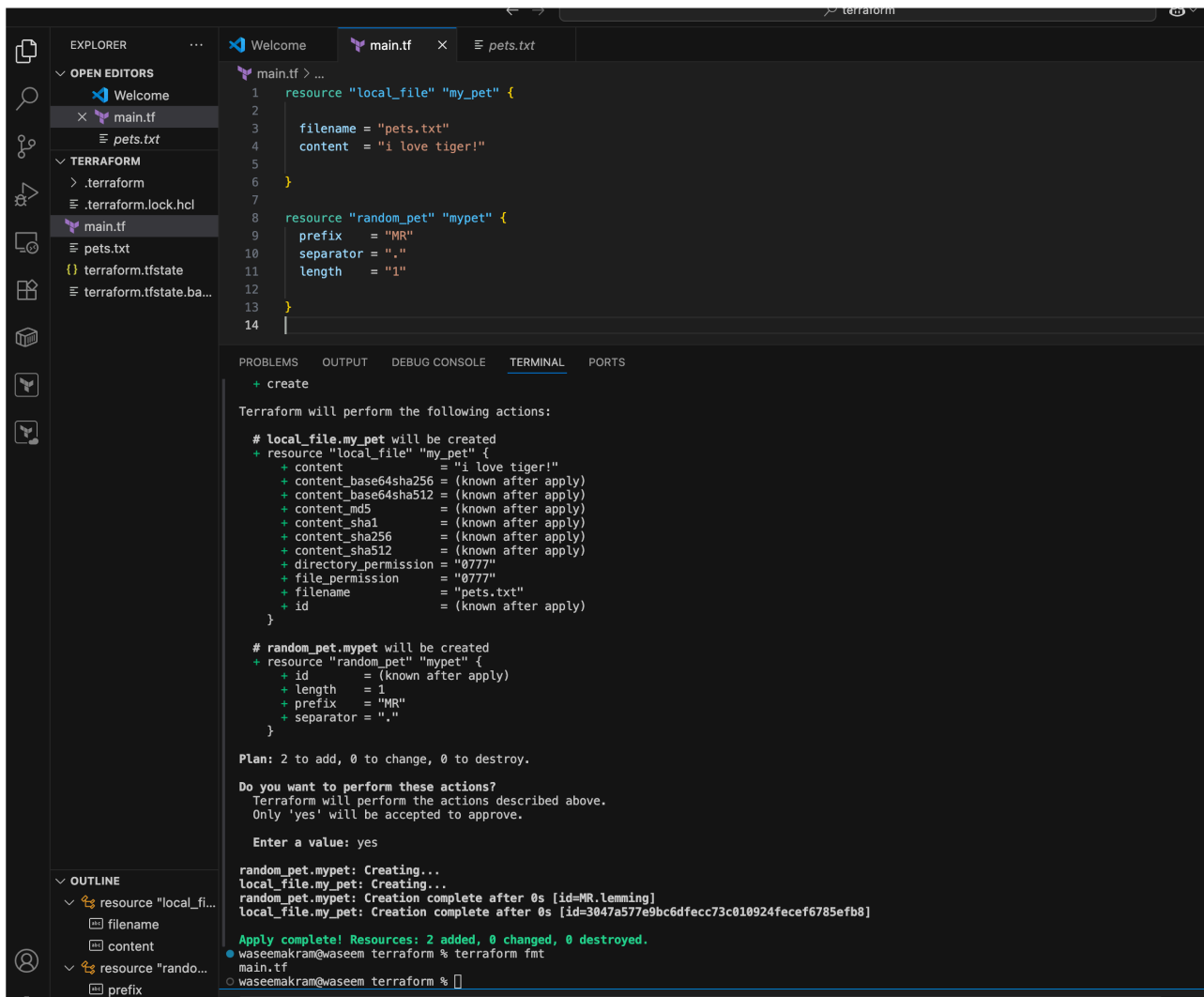
```
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
```

```
Enter a value: yes
```

```
random_pet.mypet: Creating...
local_file.my_pet: Creating...
random_pet.mypet: Creation complete after 0s [id=MR.lemming]
local_file.my_pet: Creation complete after 0s [id=3047a577e9bc6dfec73c010924fecef6785efb8]
```

```
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

```
• waseemakram@waseem terraform % terraform fmt
main.tf
○ waseemakram@waseem terraform %
```

Task : 3

3) Note down below points,

Terraform Core Concepts

3.1 Terraform Init

Terraform Init is the first command that must be executed in any Terraform project.

Purpose:

- Initializes a Terraform working directory
- Downloads required provider plugins
- Creates the `.terraform` directory

Command:

```
terraform init
```

3.2 Terraform Plan

Terraform Plan is used to preview the changes Terraform will make.

Purpose:

- Shows execution plan
- Does not create or modify resources
- Helps in reviewing changes before applying

Command:

```
terraform plan
```

3.3 Terraform Apply

Terraform Apply is used to create or modify infrastructure.

Purpose:

- Executes the actions proposed in the plan

- Creates cloud resources
- Requires confirmation before execution

Command:

terraform apply

3.4 Terraform Provider

A Terraform Provider is responsible for interacting with cloud platforms.

Purpose:

- Defines the cloud service (AWS, Azure, GCP)
- Enables Terraform to communicate with cloud APIs

Example (AWS Provider):

```
provider "aws" {  
  
  region = "us-east-1"  
  
}
```

Task :4

4).Integrate a sample Terraform template in Jenkins.

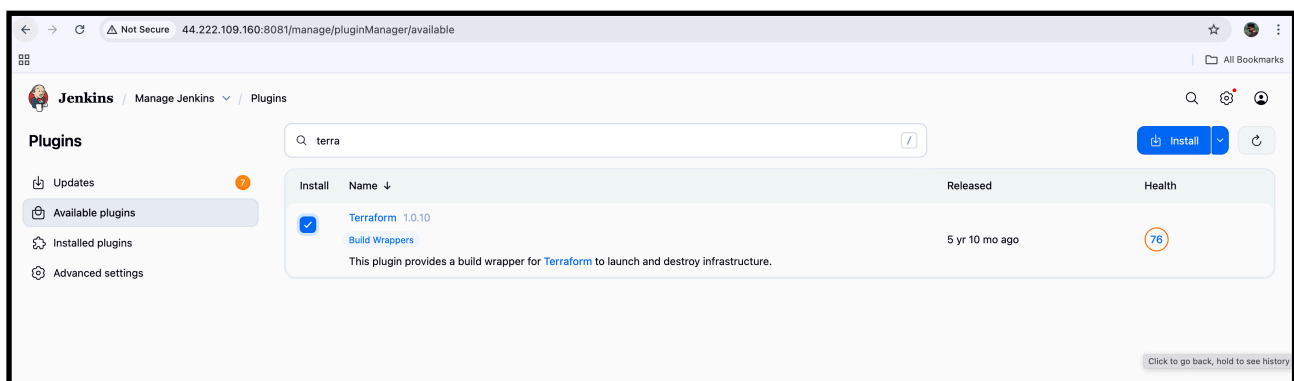
Objective :

To integrate a sample Terraform template with Jenkins so that Terraform commands can be executed automatically using a Jenkins pipeline.

Prerequisites :

1. **Jenkins server** up and running.
2. **Terraform installed** on Jenkins server (terraform -v should work).
3. Git is installed on Jenkins server
4. A **Git repo** containing Terraform code (for example: main.tf). And jenkinsfile

Install terraform plugin



Step-1 : In Jenkins server install terraform

```
wget https://releases.hashicorp.com/terraform/1.14.3/
terraform_1.14.3_linux_amd64.zip
```

```
[root@ip-172-31-65-24 ~]# wget https://releases.hashicorp.com/terraform/1.14.3/terraform_1.14.3_linux_amd64.zip
--2026-01-08 11:40:05-- https://releases.hashicorp.com/terraform/1.14.3/terraform_1.14.3_linux_amd64.zip
Resolving releases.hashicorp.com (releases.hashicorp.com)... 3.171.85.65, 3.171.85.107, 3.171.85.80, ...
Connecting to releases.hashicorp.com (releases.hashicorp.com)|3.171.85.65|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 30895477 (29M) [application/zip]
Saving to: 'terraform_1.14.3_linux_amd64.zip'

terraform_1.14.3_linux_amd64.zip      100%[=====]
2026-01-08 11:40:06 (538 MB/s) - 'terraform_1.14.3_linux_amd64.zip' saved [30895477/30895477]

[root@ip-172-31-65-24 ~]#
```

Unzip terraform_1.14.3_linux_amd64.zip

```
[root@ip-172-31-65-24 ~]# unzip terraform_1.14.3_linux_amd64.zip
Archive:  terraform_1.14.3_linux_amd64.zip
  inflating: LICENSE.txt
  inflating: terraform
[root@ip-172-31-65-24 ~]#
```

```
sudo mv terraform /usr/local/bin/
```

Terraform -version

```
Archive:  terraform_1.14.3_linux_amd64.zip
  inflating: LICENSE.txt
  inflating: terraform
[root@ip-172-31-65-24 ~]# sudo mv terraform /usr/local/bin/
[root@ip-172-31-65-24 ~]# terraform -version
Terraform v1.14.3
on linux_amd64
[root@ip-172-31-65-24 ~]#
```

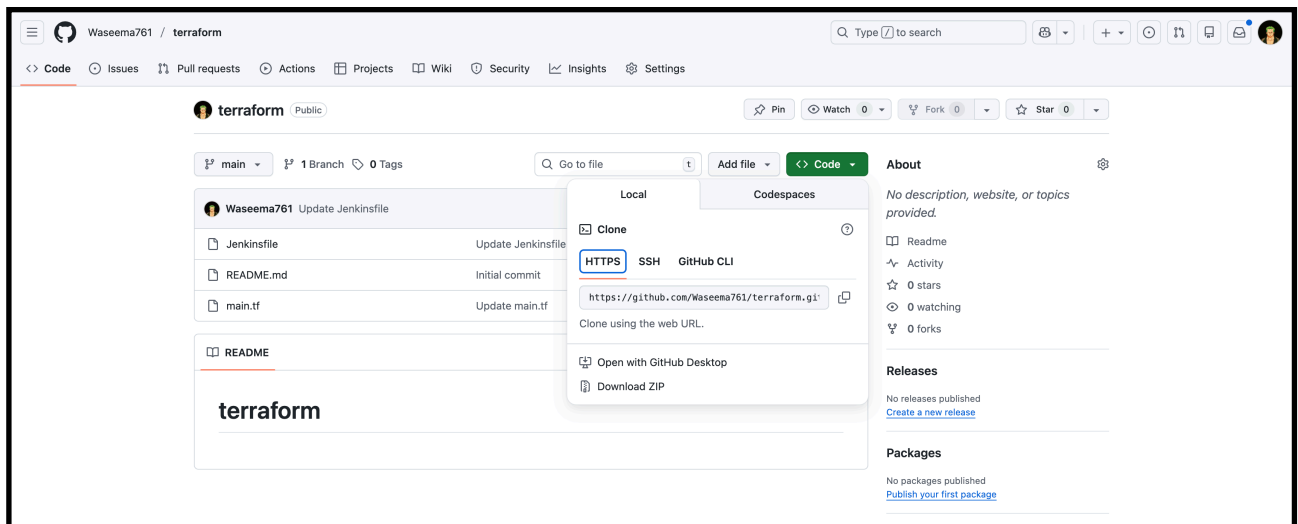
step 2:

In Jenkins tools added terraform



Step 3:

Create a New repo in git hub and and 2 new files main.tf and Jenkinfile in the git hub repo



github.com/Waseema761/terraform/blob/main/Jenkinsfile

Waseema761 / terraform

<> Code Issues Pull requests Actions Projects Wiki Security Insights

main terraform / Jenkinsfile

Waseema761 Update Jenkinsfile

Code Blame 30 lines (25 loc) · 514 Bytes

```
1 pipeline {
2   agent any
3
4   stages {
5     stage('Checkout SCM') {
6       steps {
7         checkout scm
8       }
9     }
10
11    stage('Terraform Init') {
12      steps {
13        sh 'terraform init'
14      }
15    }
16
17    stage('Terraform Plan') {
18      steps {
19        sh 'terraform plan'
20      }
21    }
22
23    stage('Terraform Apply') {
24      steps {
25        sh 'terraform apply -auto-approve'
26      }
27    }
28  }
29 }
```

github.com/Waseema761/terraform/blob/main/main.tf

Waseema761 / terraform / main.tf

Waseema761 Update main.tf

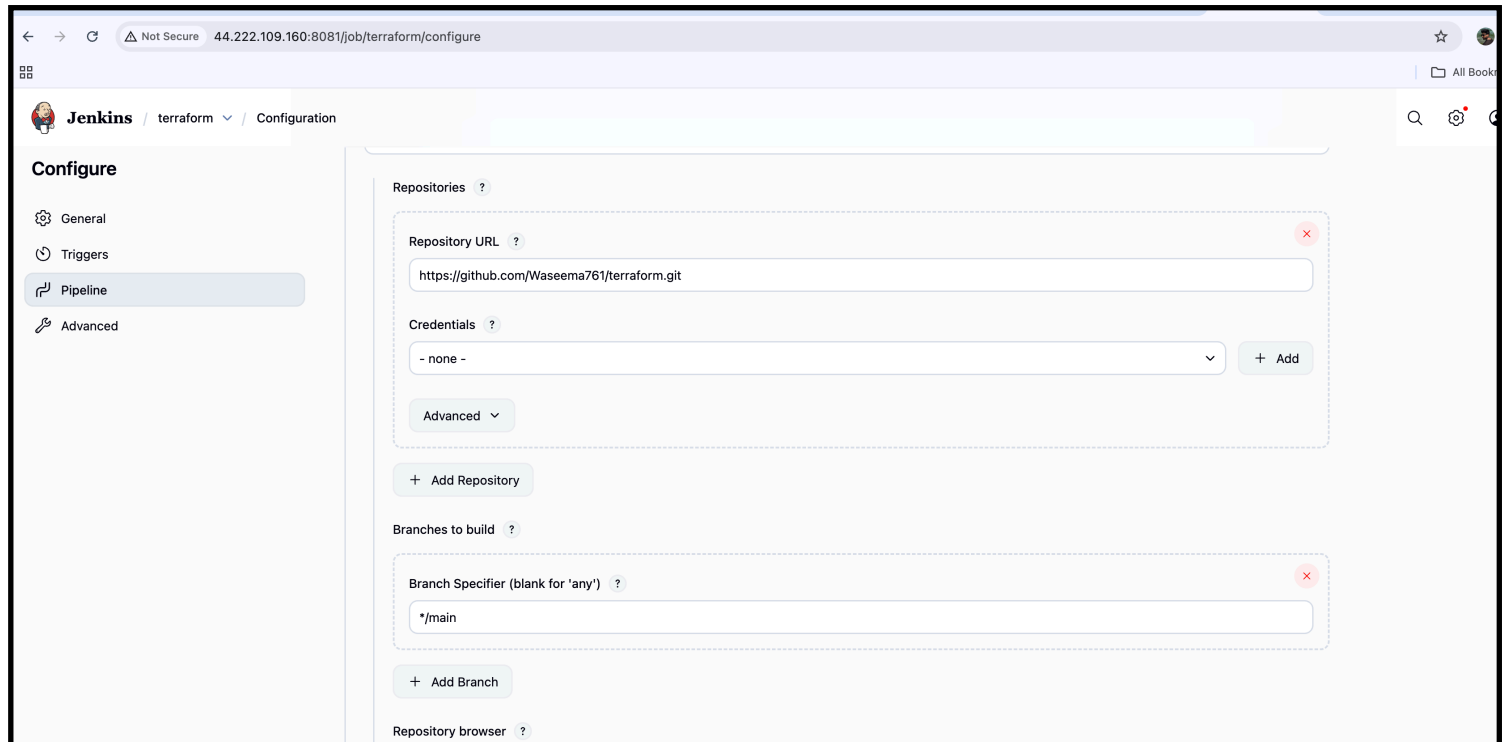
Code Blame 5 lines (4 loc) · 89 Bytes

```
1 resource "local_file" "my_pet" {
2   filename = "pets.txt"
3   content  = "i love tiger!"
4 }
```

Step 4:

Create a new item terraform job.in Jenkins

And select the pipeline and the git hub url and the branch in the configurations



The screenshot shows the Jenkins web interface for configuring a new job. The browser address bar indicates the URL is `44.222.109.160:8081/job/terraform/configure`. The left sidebar shows the 'Configure' page with tabs for 'General', 'Triggers', 'Pipeline', and 'Advanced'. The 'Pipeline' tab is selected. The main configuration area is titled 'Repositories' and contains a 'Repository URL' field with the value `https://github.com/Waseema761/terraform.git`. Below this is a 'Credentials' dropdown menu set to '- none -' with an '+ Add' button. An 'Advanced' dropdown is also visible. Below the repository section is a '+ Add Repository' button. The 'Branches to build' section contains a 'Branch Specifier (blank for \'any\')' field with the value `*/main` and an '+ Add Branch' button. At the bottom, there is a 'Repository browser' section.

—-> save the and build

Status

✓ terraform

</> Changes

▷ Build Now

⚙️ Configure

🗑️ Delete Pipeline

📁 Stages

✎️ Rename

❓ Pipeline Syntax

Permalinks

- [Last build \(#4\), 18 min ago](#)
- [Last stable build \(#4\), 18 min ago](#)
- [Last successful build \(#4\), 18 min ago](#)
- [Last failed build \(#3\), 28 min ago](#)
- [Last unsuccessful build \(#3\), 28 min ago](#)
- [Last completed build \(#4\), 18 min ago](#)

Builds

🔍 Filter

Today

✓ #4 12:13 PM

✗ #3 12:03 PM

✓ #2 12:00 PM

✗ #1 11:58 AM

Start Checkout SCM Checkout SCM Terraform Init Terraform Plan Terraform Apply End

🔍 Search

✓ Terraform Apply

🕒 0.29s

✓ Checkout SCM 0.16s

✓ Checkout SCM 0.13s

✓ Terraform Init 0.56s

✓ Terraform Plan 0.29s

✓ Terraform Apply 0.29s

✓ terraform apply -auto-approve

```
0 + terraform apply -auto-approve
1
2 Terraform used the selected providers to generate the following execution
3 plan. Resource actions are indicated with the following symbols:
4 + create
5
6 Terraform will perform the following actions:
7
8 # local_file.my_pet will be created
9 + resource "local_file" "my_pet" {
10   + content             = "i love tiger!"
11   + content_base64sha256 = (known after apply)
12   + content_base64sha512 = (known after apply)
13   + content_md5         = (known after apply)
14   + content_sha1        = (known after apply)
15   + content_sha256      = (known after apply)
16   + content_sha512      = (known after apply)
17   + directory_permission = "0777"
18   + file_permission     = "0777"
19   + filename            = "pets.txt"
20   + id                  = (known after apply)
21 }
22
23 Plan: 1 to add, 0 to change, 0 to destroy.
24 local_file.my_pet: Creating...
```

Verification :

Now lets check in Jenkins workspace

```
[root@ip-172-31-65-24 jenkins]# cd workspace/
[root@ip-172-31-65-24 workspace]# ls
'MULTI-STAGE PIPELINE'      hiring-app-declarative      job2                         pipeline                    'scripted pipeline'        simplecustomerapp-pipeline  vprofile
'MULTI-STAGE PIPELINE@tmp'  hiring-app-declarative@tmp  job3                       pipeline@tmp               'scripted pipeline@tmp'    simplecustomerapp-pipeline@tmp
first-job                  hiring-app-parallel         parametirized-job          private                    scripted-pipeline1         terraform
hiring-app                 hiring-app-parallel@tmp     parametirized-job@tmp     private@tmp               scripted-pipeline1@tmp     terraform@tmp
[root@ip-172-31-65-24 workspace]# cd terraform
[root@ip-172-31-65-24 terraform]# ls
Jenkinsfile  README.md  main.tf  pets.txt  terraform.tfstate
[root@ip-172-31-65-24 terraform]#
```

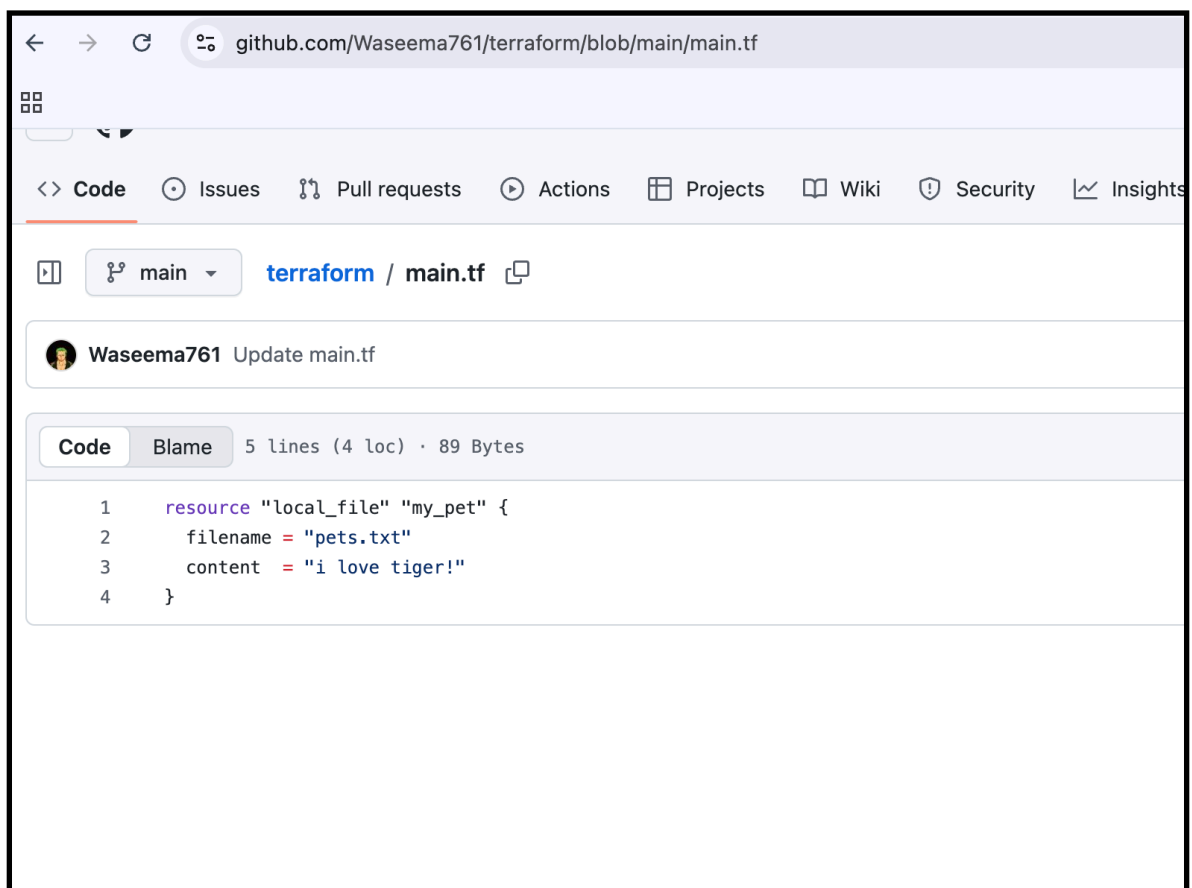
We are able to see the pets.txt file

—> now to confirm lets cat the file

```
Jenkinsfile  README.md  main.tf  pets.txt  ter
[root@ip-172-31-65-24 terraform]# cat pets.txt
i love tiger!
[root@ip-172-31-65-24 terraform]#
```

```
[root@ip-172-31-65-24 terraform]# cat pets.txt
i love tiger!
```

—> same as given below code from GitHub



The screenshot shows a GitHub repository page for 'Waseema761/terraform'. The 'Code' tab is selected, displaying the content of 'main.tf'. The file contains a Terraform resource definition for a local file named 'my_pet' with the filename 'pets.txt' and the content 'i love tiger!'.

```
1 resource "local_file" "my_pet" {
2     filename = "pets.txt"
3     content  = "i love tiger!"
4 }
```

